

Date –

Experiment – 06

Aim - Exercises to plot the functions and to find its first and second derivatives in Scilab.

Code

```
% Define x-axis range
x = linspace(1, 250, 15);

% Calculate function values at each x point
y = f(x);

% Plot the function
plot(x, y);
xlabel('x');
ylabel('f(x)');
title('Plot of f(x)');
grid on;
hold on;

% Calculate first derivative values at each x point
dy = df(x);

% Plot the first derivative (optional)
% plot(x, dy);

% Print the first derivative at a specific point
x_point = 2; % Example point
first_derivative = df(x_point);
disp('First derivative at x=2:');
disp(first_derivative);

% Calculate second derivative values at each x point
d2y = d2f(x);

% Plot the second derivative (optional)
% plot(x, d2y);

% Print the second derivative at a specific point
second_derivative = d2f(x_point);
```

```

disp('Second derivative at x=2:');
disp(second_derivative);

% Define function f(x)
function y = f(x)
    y = x.^2 + sin(x); % Use .^ for element-wise power
end

% Define the first derivative function df(x)
function dy = df(x)
    dy = 2*x + cos(x);
end

% Define the second derivative function d2f(x)
function d2y = d2f(x)
    d2y = 2 - sin(x);
end

```

Output

Scilab 2023.0.0 Console

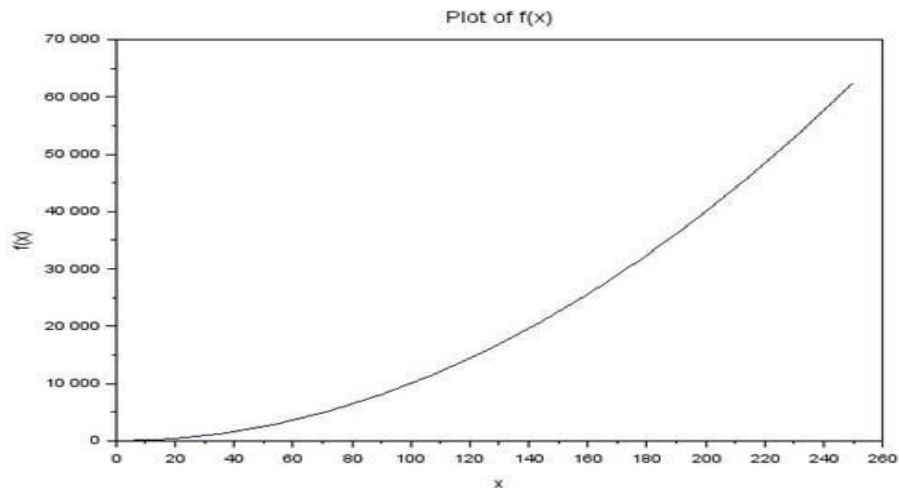
```
--> exec('S:\SSMDA Lab\Program-6', -1)
```

```
"First derivative at x=2:"
```

```
3.5838532
```

```
"Second derivative at x=2:"
```

```
1.0907026
```



Date -

Experiment – 14

Aim - Write a program to implement concepts of probability and distributions in R.

Code

```
##### Binomial Distribution #####
# Probability of getting exactly 5 successes in 8 trials with success prob 1/6
dbinom(x = 5, size = 8, prob = 1/6)

# Probability mass function for all outcomes from 0 to 8
x_prob_binomial <- dbinom(x = 0:8, size = 8, prob = 1/6)
print(x_prob_binomial)
print(sum(x_prob_binomial)) # should be ~1
print(round(x_prob_binomial, 3))

# Barplot
barplot(x_prob_binomial, names.arg = 0:8, space = 0, xlab = "x", ylab = "Pr(X = x)", col =
"green")

# CDF: Pr(X <= 3)
pbinom(q = 3, size = 8, prob = 1/6)

##### Poisson Distribution #####
# Probability Pr(X = 3) when lambda = 3.22
dpois(x = 3, lambda = 3.22)
dpois(x = 0, lambda = 3.22)
round(dpois(0:10, 3.22), 3)

# Barplot
barplot(dpois(x = 0:10, lambda = 3.22), ylim = c(0, 0.25), space = 0, names.arg = 0:10, ylab =
"Pr(X=x)", xlab = "x", col = "blue")

# CDF: Pr(X <= 2)
ppois(q = 2, lambda = 3.22)

##### Uniform Distribution #####
min_val <- 0
max_val <- 100
xpos <- seq(min_val, max_val, by = 0.5)
```

```

ypos <- dunif(xpos, min = 10, max = 80)
plot(ypos, type = "o", main = "PDF of Uniform Distribution")

# CDF
min_val <- 0
max_val <- 60
punif(15, min = min_val, max = max_val)

##### Normal Distribution #####
x <- seq(-15, 10, length.out = 100)
pdf <- dnorm(x, mean = mean(x), sd = sd(x))
plot(x, pdf, type = "l", main = "PDF of Normal Distribution")

# CDF using ecdf
cdf <- ecdf(x)
plot(cdf, main = "CDF Graph", xlab = "x", ylab = "y")

# CDF using gbutils
# install.packages("gbutils") # uncomment if not installed
library(gbutils)
cdf1 <- pnorm(x, mean = -2.5, sd = 7.64)
plotpdf(cdf1, cdf = cdf1, main = "CDF Plot")

##### QQ-Plot #####
norm_samp <- rnorm(100)
qqnorm(norm_samp)
abline(a = 0, b = 1, col = "grey")

##### Student's t-distribution #####
x <- seq(-6, 6, length = 100)
df <- c(1, 4, 10, 30)
colour <- c("red", "orange", "green", "yellow", "black")

# Plot normal distribution
plot(x, dnorm(x), type = "l", lty = 2, xlab = "t-value", ylab = "Density", main = "Comparison of
t-distributions", col = "black")

# Add t-distributions
for (i in 1:4) {
  lines(x, dt(x, df[i]), col = colour[i])
}

```

```
}
```

```
# Legend
```

```
legend("topright", c("df = 1", "df = 4", "df = 10", "df = 30", "normal"), col = c(colour[1:4],  
"black"), title = "t-distributions", lty = c(1, 1, 1, 1, 2))
```

```
##### Chi-square distribution #####
```

```
df_chi <- 5
```

```
vec <- 0:4
```

```
# PDF and CDF
```

```
print("Calculating for the values [0, 4]")
```

```
print(dchisq(vec, df = df_chi))
```

```
print(pchisq(4, df = df_chi, lower.tail = TRUE))
```

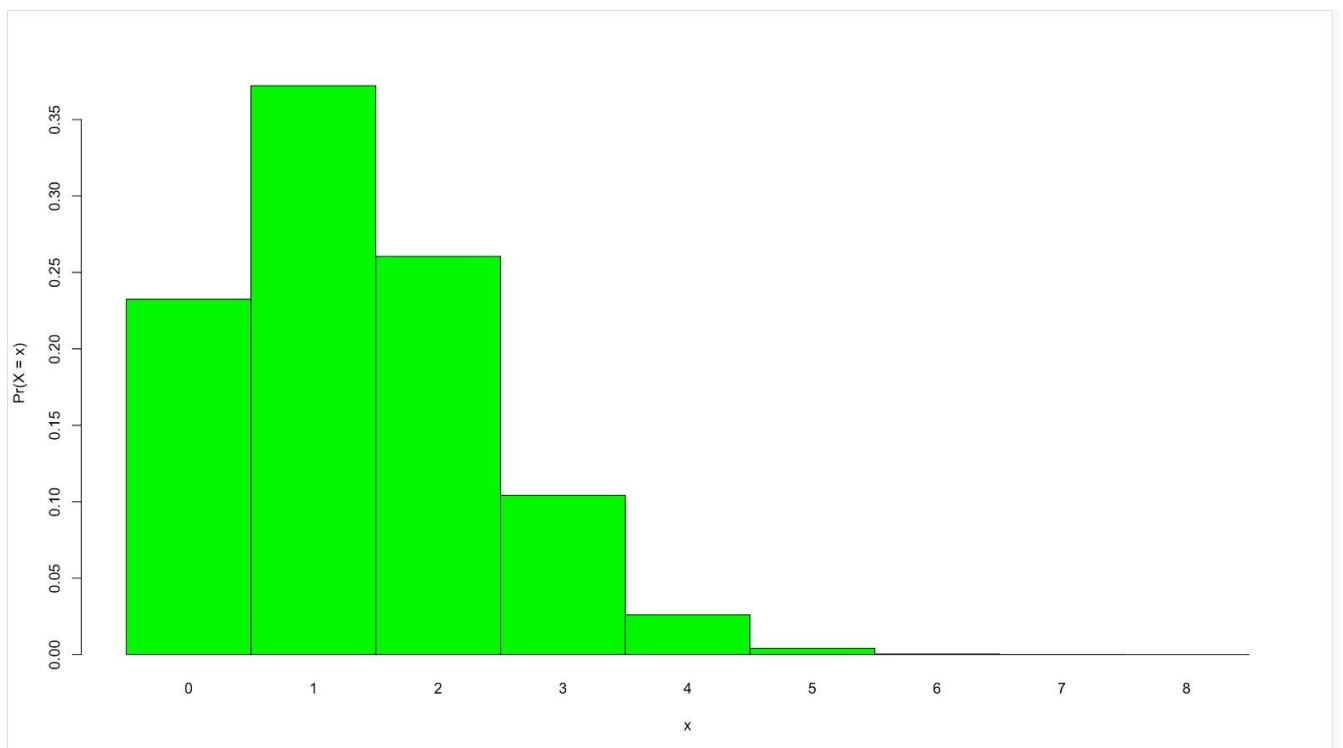
```
# Histogram and curve
```

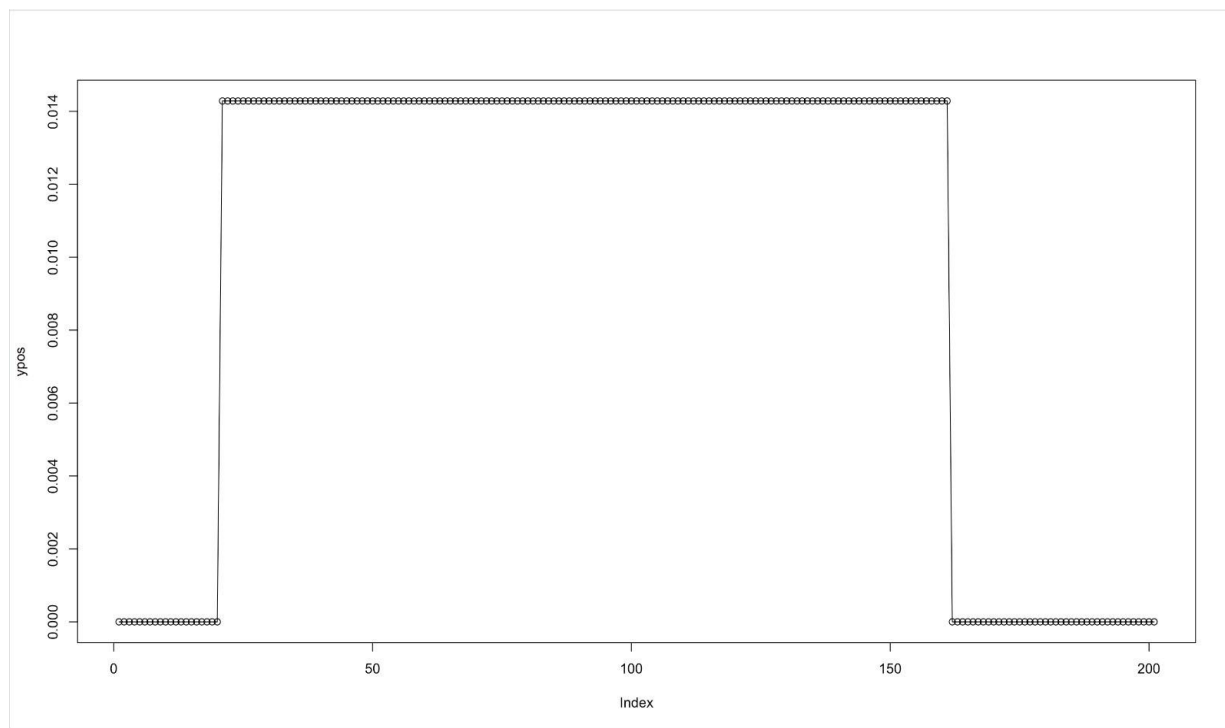
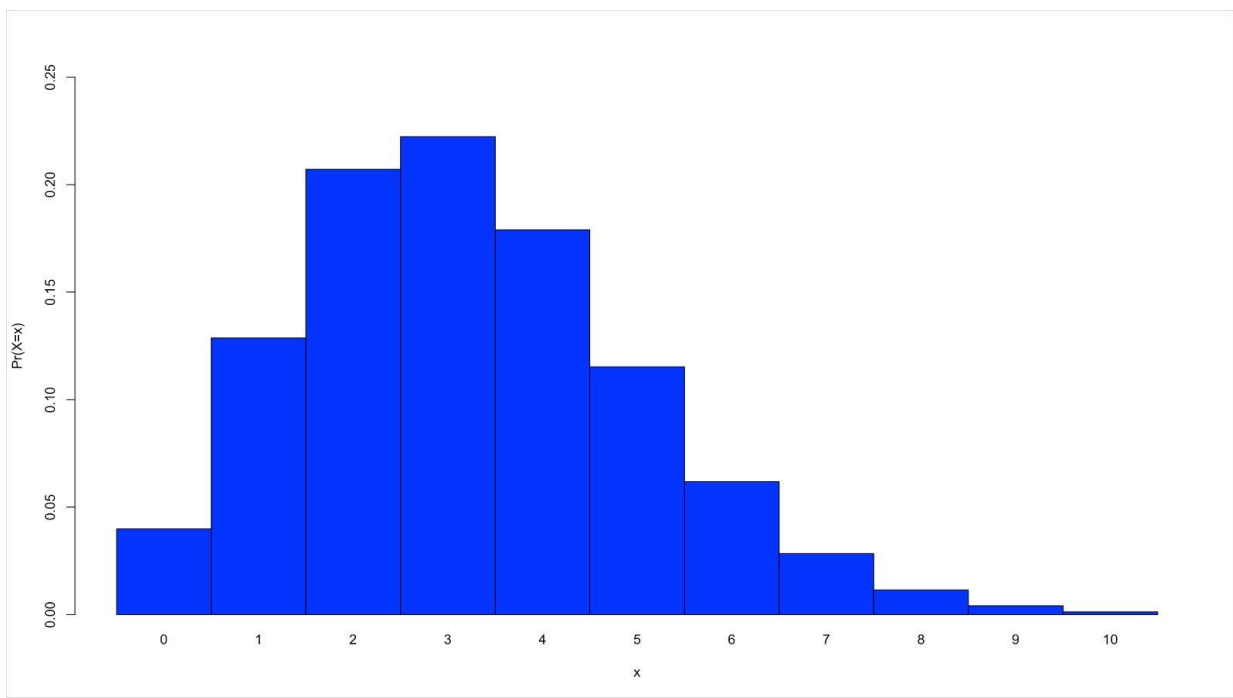
```
x <- rchisq(50000, df = 4)
```

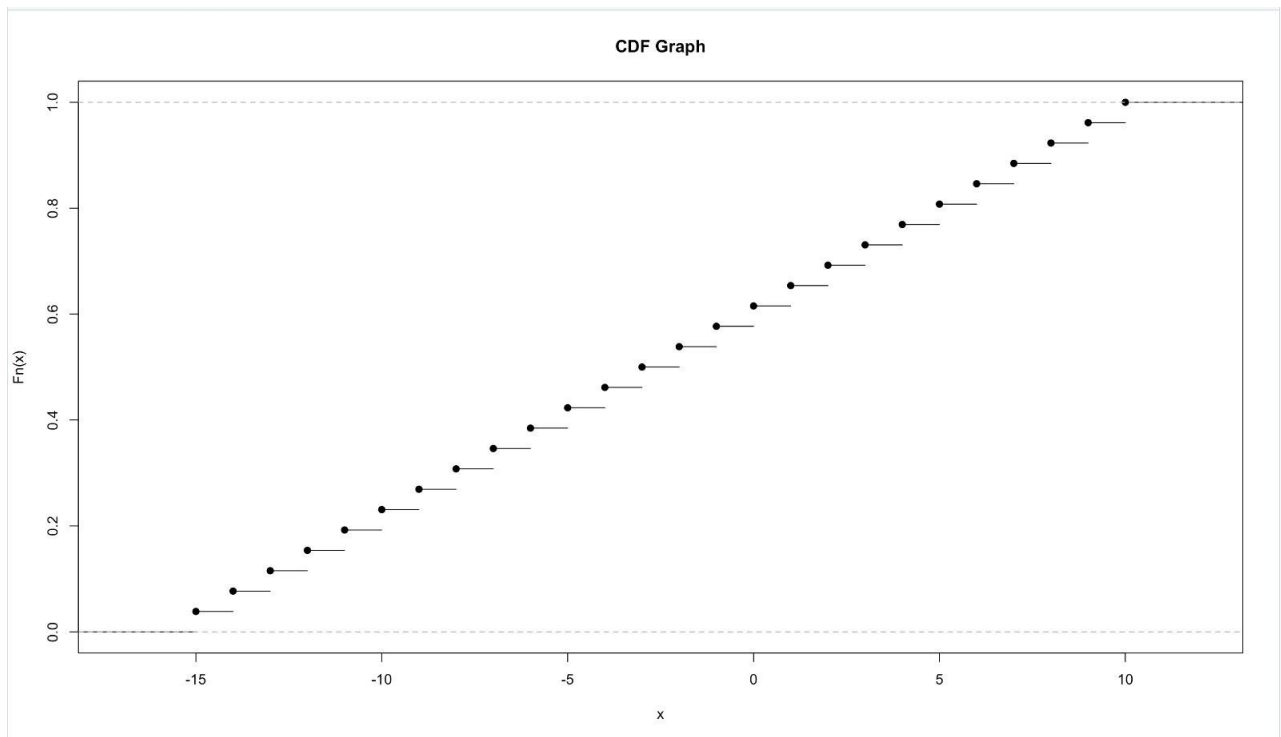
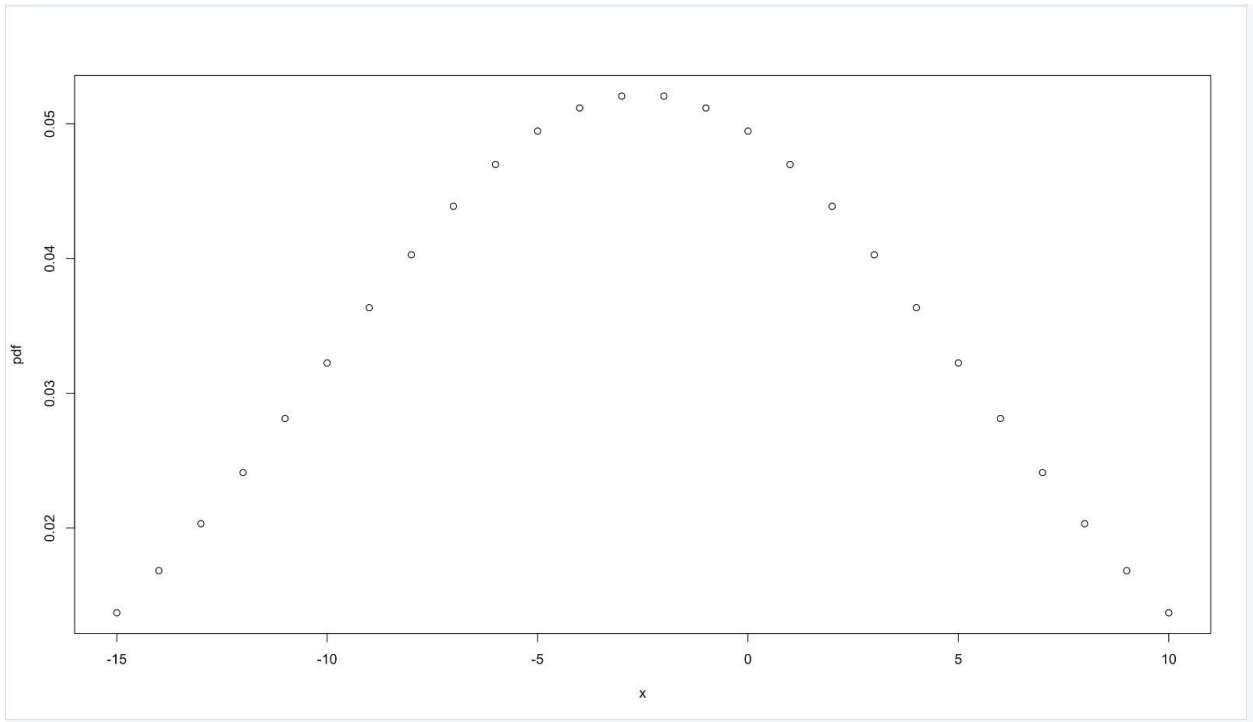
```
hist(x, freq = FALSE, xlim = c(0, 16), ylim = c(0, 0.2), col = 'gray', main = "Chi-square  
Distribution with df=4")
```

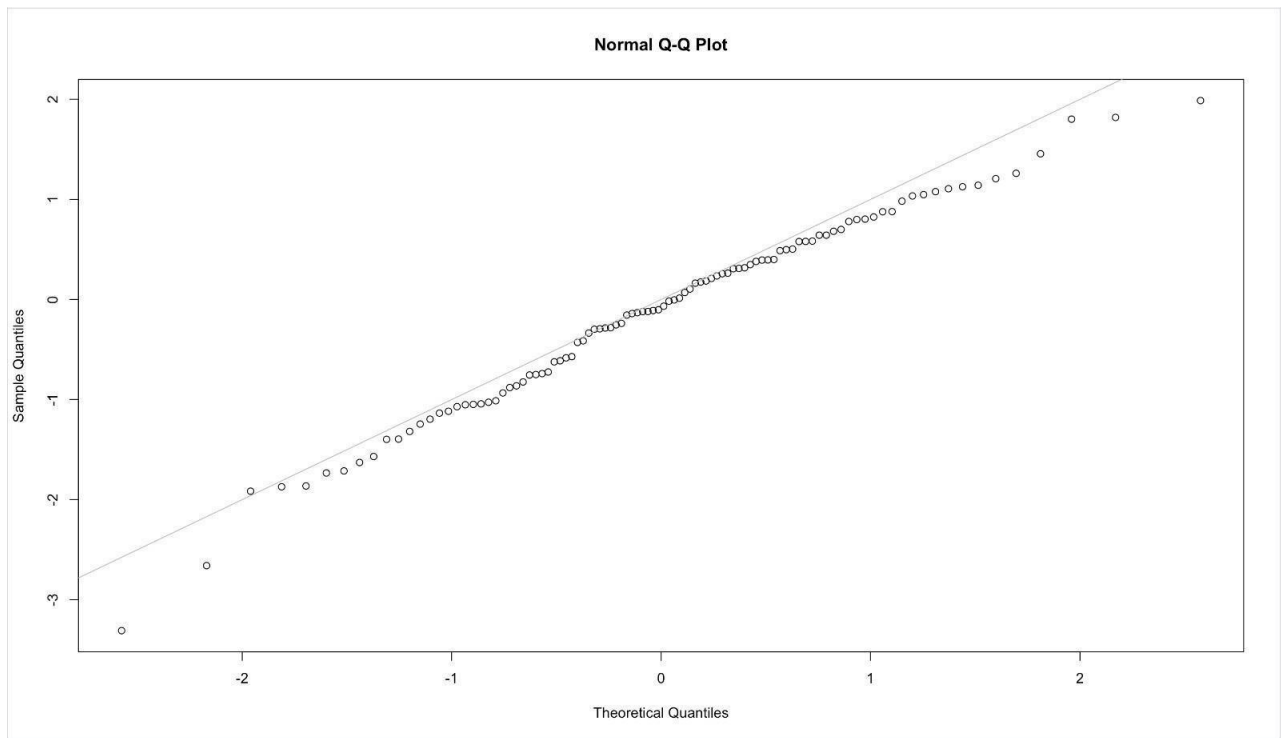
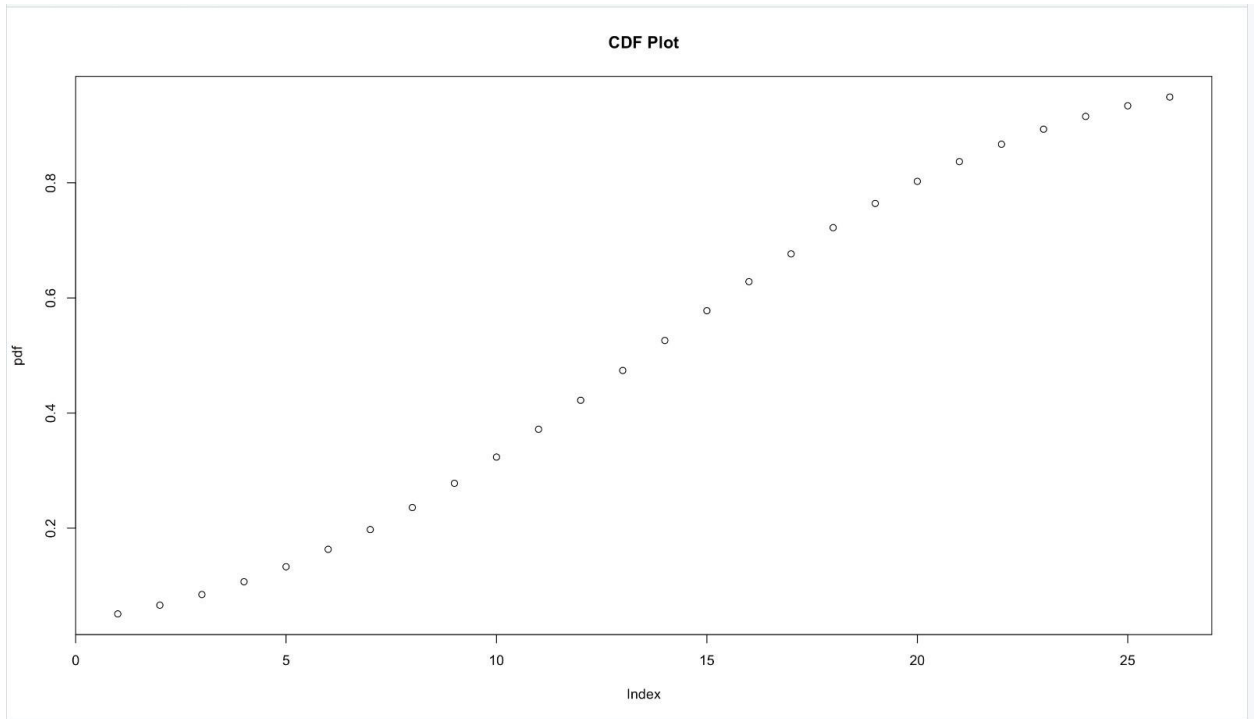
```
curve(dchisq(x, df = 4), from = 0, to = 15, n = 5000, col = 'red', lwd = 2, add = TRUE)
```

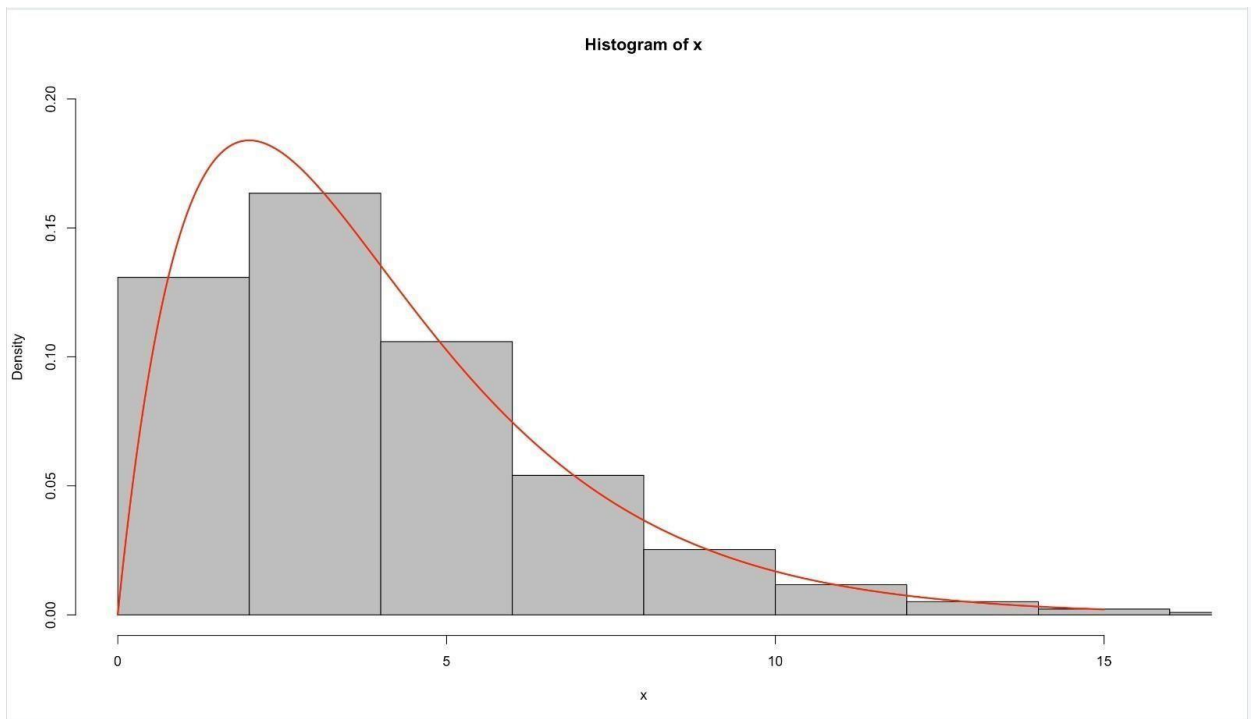
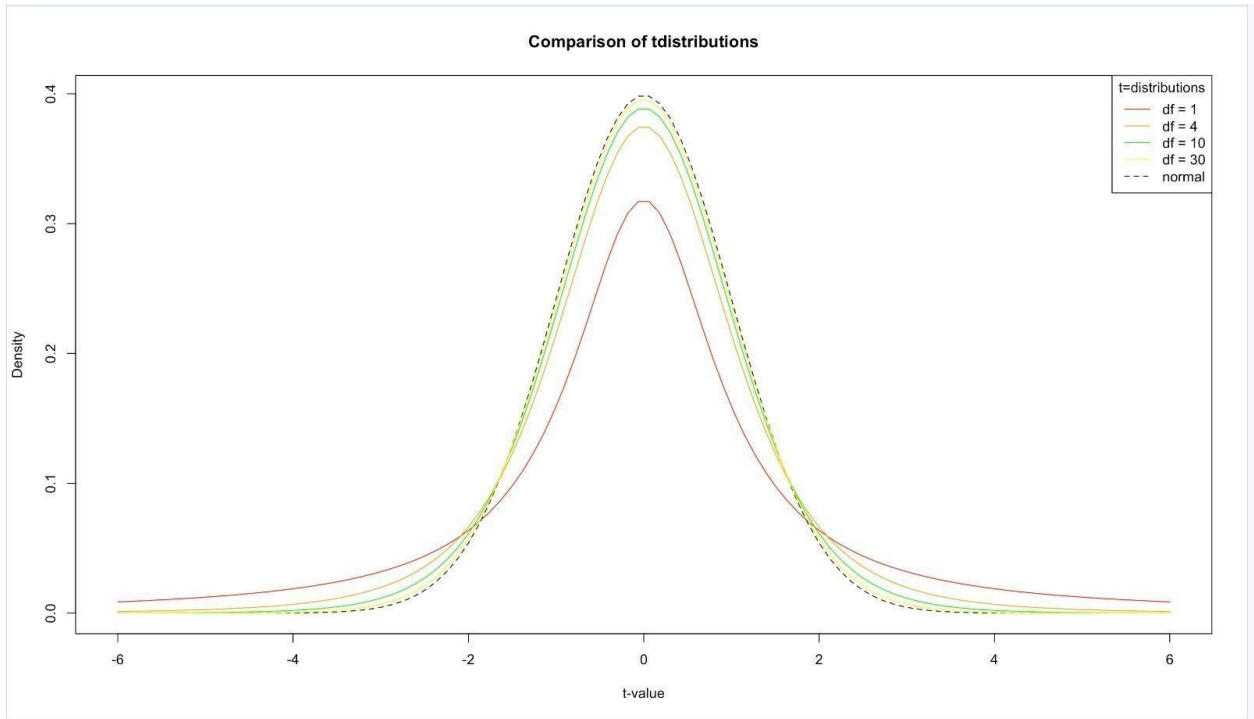
Output











Date –

Experiment – 13

Aim - Write a program to implement linear regression using R.

Code

```
# Load the dataset
data(mtcars)

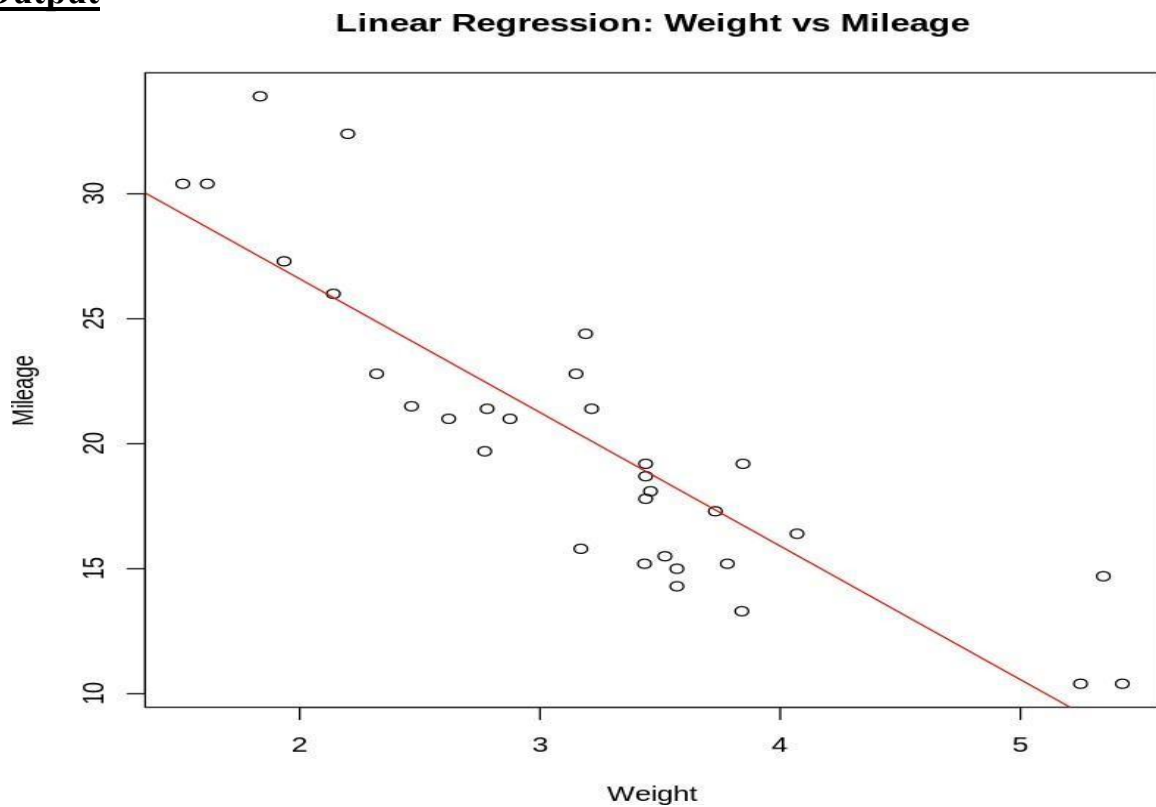
# Assign variables
X <- mtcars$wt
Y <- mtcars$mpg

# Fit a linear regression model
model <- lm(Y ~ X)

# Plot the scatter plot
plot(X, Y,
     xlab = "Weight",
     ylab = "Mileage",
     main = "Linear Regression: Weight vs Mileage")

# Add the regression line
abline(model, col = "red")
```

Output



Date –

Experiment – 12

Aim - Write a program to implement Time series Analysis using R.

Implementing Time Series Analysis in R:

Step 1 - Load the dataset

```
data("AirPassengers")  
print(head(AirPassengers)) # View first few entries
```

Step 2 - Check structure

```
str(AirPassengers) # Confirm it's a 'ts' object
```

Step 3 - Plot the data

```
plot(AirPassengers,  
     main = "Monthly Air Passengers (1949-1960)",  
     ylab = "Passengers (in thousands)",  
     xlab = "Year")
```

Step 4 - Decompose the time series

```
decomposed <- decompose(AirPassengers)  
plot(decomposed)
```

Step 5 - Forecasting future data

```
install.packages("forecast") # Only once  
library(forecast)  
forecast_data <- forecast(AirPassengers, h = 24)  
plot(forecast_data)
```

```
print(Nile)          # Prints all values  
length(Nile)         # Total number of observations  
head(Nile, n = 10)    # First 10 values  
tail(Nile, n = 12)    # Last 12 values
```

```
# Plot the time series
```

```
plot(Nile,  
     col = "blue",  
     xlab = "Year",  
     ylab = expression("River Volume (10^9* m^3)"),  
     main = "Annual Flow of the Nile River")
```

Output

```
Source

Console Background Jobs x
R 4.3.3 · ~/
> source("~/Documents/Files/Stats/Program7.R", echo=TRUE)

> ## Air Passengers dataset ##
> # Load the dataset
> data("AirPassengers")

> # Print the first few rows
> print(head(AirPassengers))
      Jan Feb Mar Apr May Jun
1949 112 118 132 129 121 135

> # Check the structure
> str(AirPassengers)
Time-Series [1:144] from 1949 to 1961: 112 118 132 129 121 135 148 148 136 119 ...

> # Plot the dataset
> plot(AirPassengers)

> # Decompose the time series data
> decomposed <- decompose(AirPassengers)

> # Plot the decomposed data
> plot(decomposed)

> # Load the 'forecast' package
> library(forecast)

> # Forecast future data
> forecast_data <- forecast(AirPassengers, h = 24)

> # Plot the forecasted data
> plot(forecast_data)

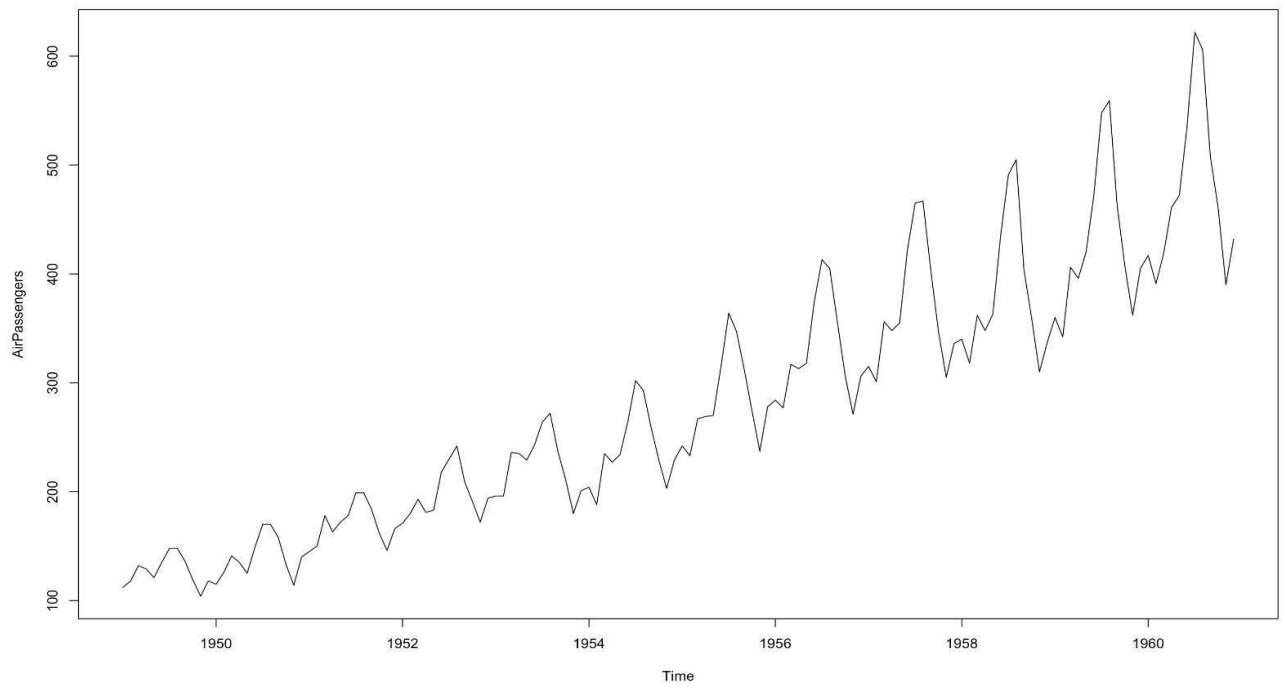
> #####
> # Print the Nile dataset
> ##### .... [TRUNCATED]
Time Series:
Start = 1871
End = 1970
Frequency = 1
[1] 1120 1160 963 1210 1160 1160 813 1230 1370 1140 995 935 1110 994 1020 960 1180 799 958 1140 1100 1210 1150 1250
[25] 1260 1220 1030 1100 774 840 874 694 940 833 701 916 692 1020 1050 969 831 726 456 824 702 1120 1100 832
[49] 764 821 768 845 864 862 698 845 744 796 1040 759 781 865 845 944 984 897 822 1010 771 676 649 846
[73] 812 742 801 1040 860 874 848 890 744 749 838 1050 918 986 797 923 975 815 1020 906 901 1170 912 746
[97] 919 718 714 740
```

```
> length(Nile)
[1] 100

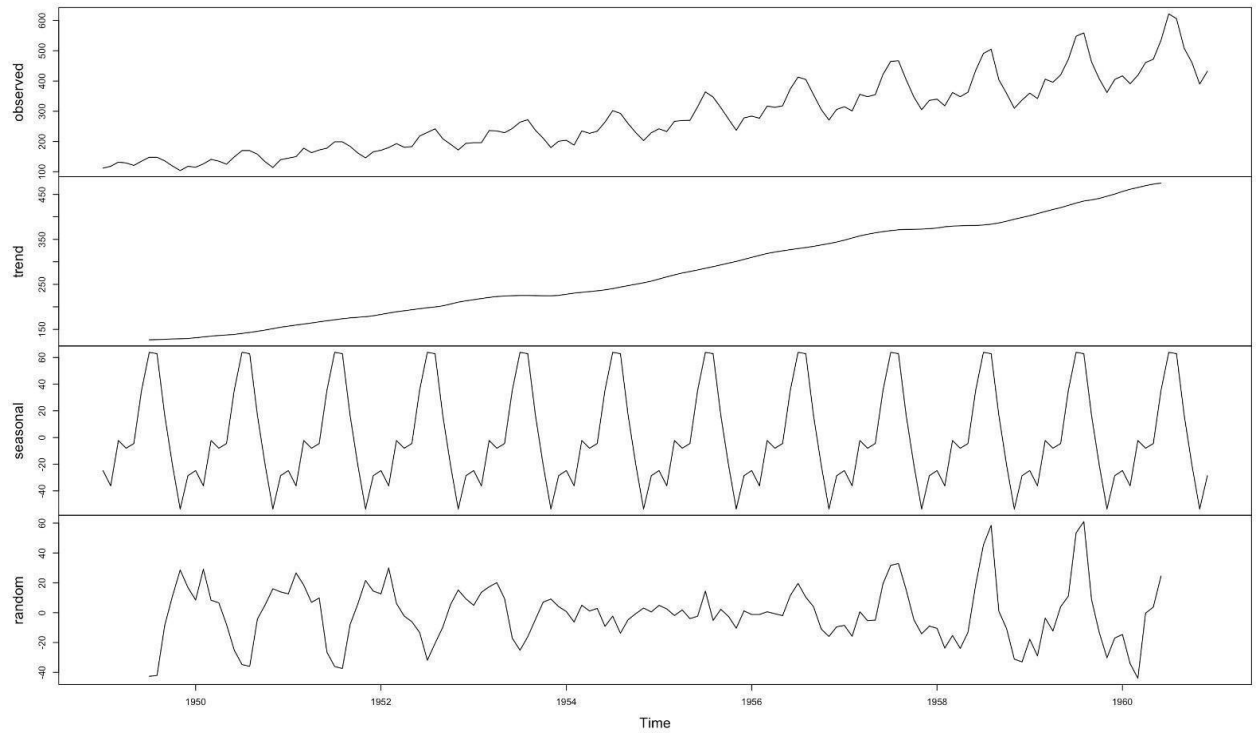
> # Display the first 10 elements of the Nile dataset
> head(Nile, n = 10)
Time Series:
Start = 1871
End = 1880
Frequency = 1
[1] 1120 1160 963 1210 1160 1160 813 1230 1370 1140

> # Display the last 12 elements of the Nile dataset
> tail(Nile, n = 12)
Time Series:
Start = 1959
End = 1970
Frequency = 1
[1] 975 815 1020 906 901 1170 912 746 919 718 714 740

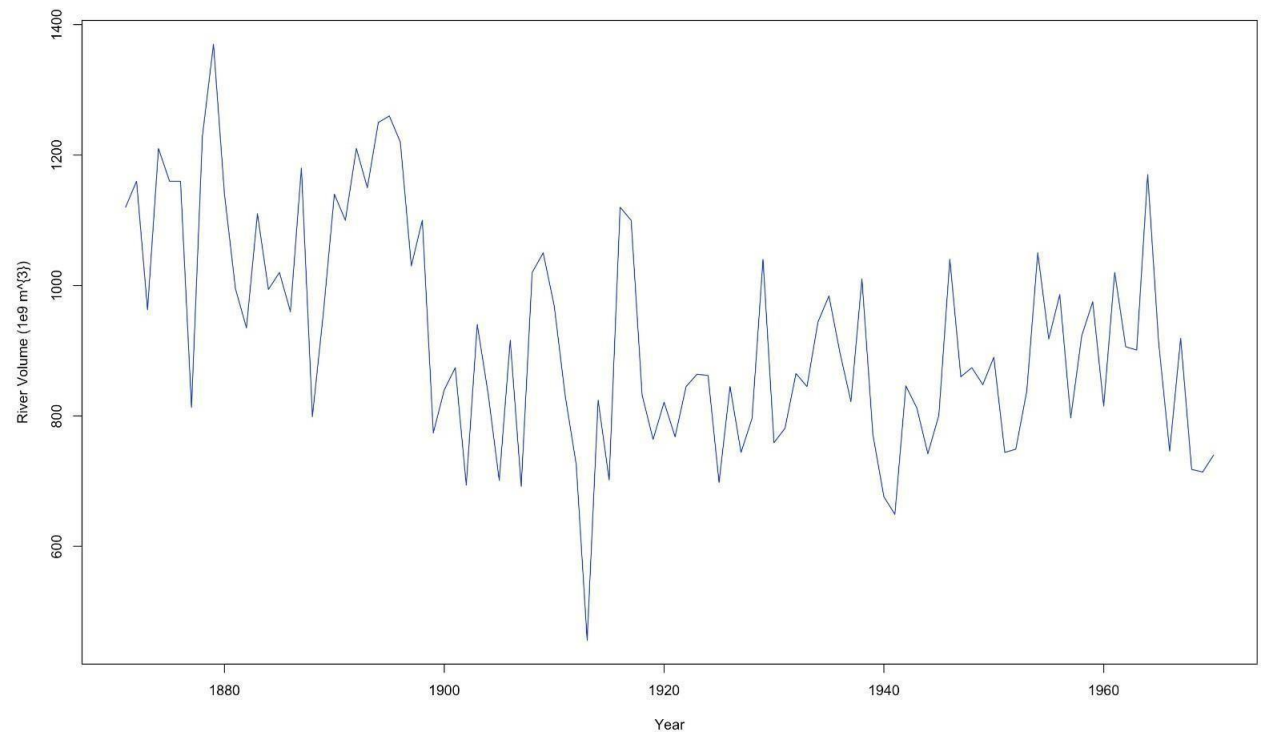
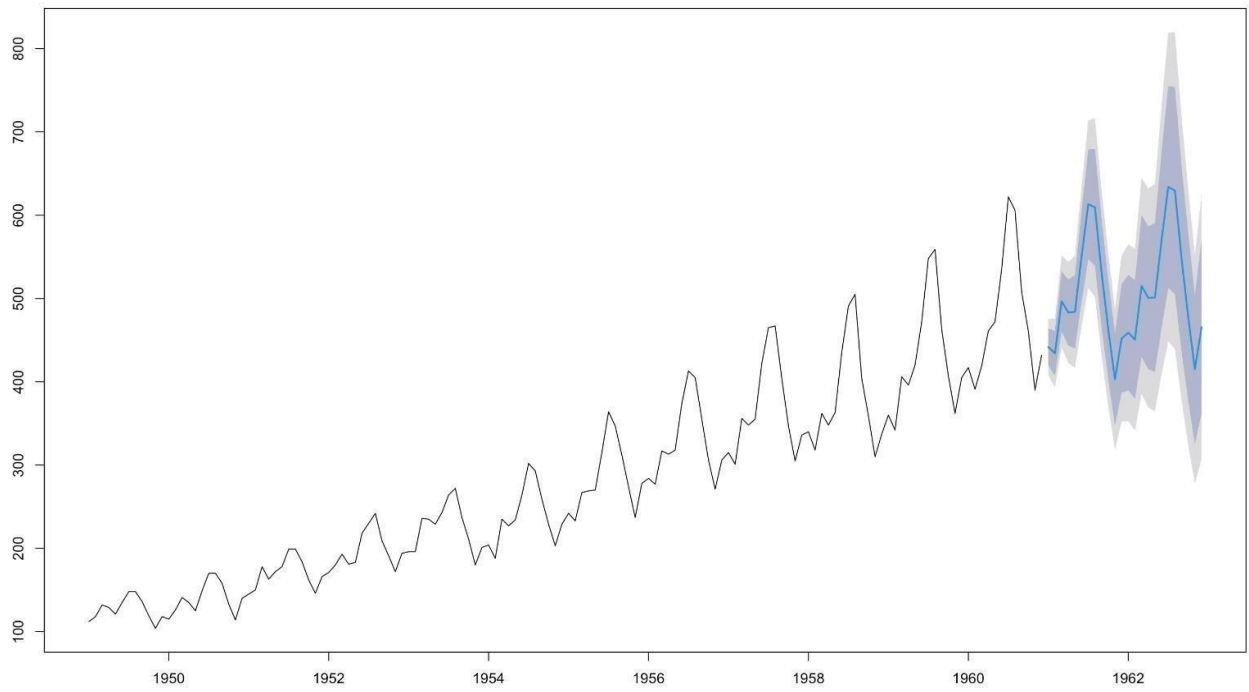
> #plot(Nile, col = "blue")
> plot(Nile,col = "blue", xlab = "Year", ylab = "River Volume (1e9 m^3)")
> |
```



Decomposition of additive time series



Forecasts from ETS(M,Ad,M)



Date –

Experiment – 11

Aim – Write a program to calculate Correlation using R.

Code

```
#### Pearson's Correlation Coefficient ####

# Define two numeric vectors
xdata <- c(2, 4.4, 3, 3, 2, 2.2, 2, 4)
ydata <- c(1, 4.4, 1, 3, 2, 2.2, 2, 7)

# Compute covariance
cat("The covariance is:\n")
print(cov(xdata, ydata))

# Compute correlation manually using covariance and standard deviations
cat("The correlation coefficient (rho) using cov and sd functions:\n")
rho <- cov(xdata, ydata) / (sd(xdata) * sd(ydata))
print(rho)

# Compute correlation directly using cor()
cat("The correlation using cor() function:\n")
print(cor(xdata, ydata))

# Scatter plot of x and y
plot(xdata, ydata, pch = 13, cex = 1.5,
     main = "Scatterplot of xdata vs ydata",
     xlab = "xdata", ylab = "ydata")

#### Correlation Matrix and Visualizations ####

# Load built-in dataset
data(mtcars)

# Load necessary libraries
library(corrplot)
library(rstatix)
library(PerformanceAnalytics)
# Uncomment below if using 'lares'
# library(lares)

# Basic correlation matrix with corrplot
```

```

corr_matrix <- cor(mtcars)
corrplot(corr_matrix, method = "circle", type = "upper",
          tl.col = "black", addCoef.col = "black", number.cex = 0.7,
          main = "Correlation Matrix (mtcars)")

# Using rstatix to perform correlation tests
cor_test <- cor_mat(mtcars)
print(cor_test)

# Using PerformanceAnalytics to visualize correlation with histograms and p-values
chart.Correlation(mtcars, histogram = TRUE, pch = 19)

# Optional: Using 'lares' package for ranked correlations
# If not installed, run: install.packages("lares")
# Ranked correlation heatmap for top correlated variables
# corr_cross(mtcars, rm.na = TRUE, max_pvalue = 0.05, top = 15, grid = TRUE)

```

Output

```

R 4.3.3 ~ /
> source("~/Documents/Files/Stats/Program3.R", echo=TRUE)

> #The program evaluates Pearson's correlation coefficient, rho, using covariance and standard deviation, and directly using correlation function.
> x .... [TRUNCATED]

> ydata <- c(1,4,4,1,3,2,2,2,7)

> print('The covariance is:')
[1] "The covariance is:"

> print(cov(xdata,ydata))
[1] 1.479286

> print('The correlation coefficient, rho, using cov function is:')
[1] "The correlation coefficient, rho, using cov function is:"

> rho = cov(xdata,ydata)/(sd(xdata)*sd(ydata)) # Computes the correlation using cov (covariance)

> # and sd (standard deviation) function
> print(rho)
[1] 0.7713962

> print('The correlation using cor function is:')
[1] "The correlation using cor function is:"

> print(cor(xdata,ydata)) # Computes the correlation coefficient using cor function
[1] 0.7713962

> #We can plot these bivariate observations as a coordinate-based plot (a scatterplot).
> #Executing the following gives figure between x and y data p .... [TRUNCATED]

> #pch symbols
> ## Creating some correlation matrix with predefined packages and dataset.
> #Below depicts sample values-to-sample values correlation .... [TRUNCATED]

> library(corrplot) #We are using the corrplot library

> #To make the correlation matrix plot -
> corrplot(cor(mtcars)) #it creates the correlation matrix.

> # In the above correlation plot, the circles and color keys represents the "correlation coefficient"
> #or value of "rho" or the degree of the linea .... [TRUNCATED]

> cor_test <- cor_mat(mtcars) #to create the correlation matrix

```

```

> cor_test
# A tibble: 11 x 12
  rowname mpg   cyl  disp    hp  drat    wt  qsec    vs    am  gear  carb
* <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 mpg     1   -0.85 -0.85 -0.78  0.68  -0.87  0.42  0.66  0.6  0.48 -0.55
2 cyl    -0.85  1    0.9  0.83 -0.7  0.78  -0.59 -0.81 -0.52 -0.49  0.53
3 disp   -0.85  0.9   1    0.79 -0.71  0.89  -0.43 -0.71 -0.59 -0.56  0.39
4 hp     -0.78  0.83  0.79  1    -0.45  0.66  -0.71 -0.72 -0.24 -0.13  0.75
5 drat    0.68 -0.7  -0.71 -0.45  1    -0.71  0.091  0.44  0.71  0.7  -0.091
6 wt     -0.87  0.78  0.89  0.66 -0.71  1    -0.17 -0.55 -0.69 -0.58  0.43
7 qsec    0.42 -0.59 -0.43 -0.71  0.091 -0.17  1    0.74 -0.23 -0.21 -0.66
8 vs      0.66 -0.81 -0.71 -0.72  0.44  -0.55  0.74  1    0.17  0.21 -0.57
9 am      0.6  -0.52 -0.59 -0.24  0.71  -0.69 -0.23  0.17  1    0.79  0.058
10 gear   0.48 -0.49 -0.56 -0.13  0.7  -0.58 -0.21  0.21  0.79  1    0.27
11 carb  -0.55  0.53  0.39  0.75 -0.091  0.43 -0.66 -0.57  0.058  0.27  1

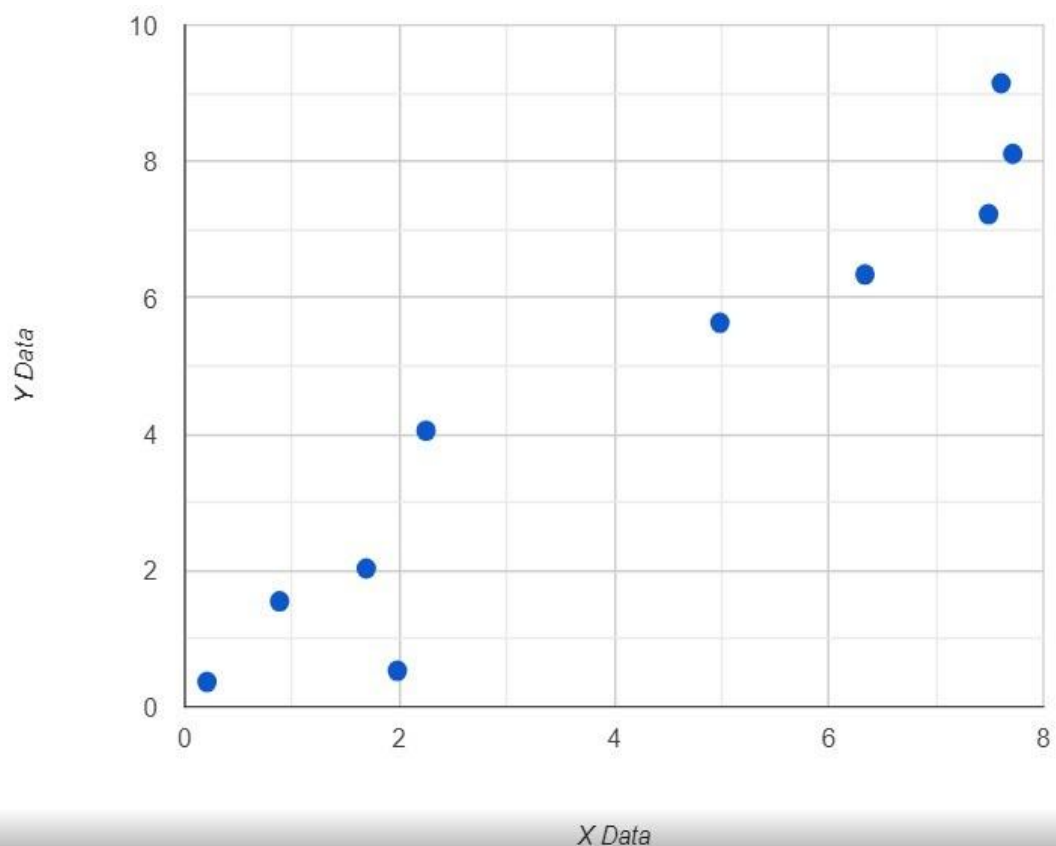
> ## Another informative is "PerformanceAnalytics", which gives the p-value,
> ## distribution (histograms), and correlation coefficient
> library(Per .... [TRUNCATED]

> chart.Correlation(cor(mtcars))

> # In the above chart, the red stars define different levels of significance, i.e. * = 0.05, ** = 0.01, *** = 0.001
> ## Using package called "Lares" .... [TRUNCATED]

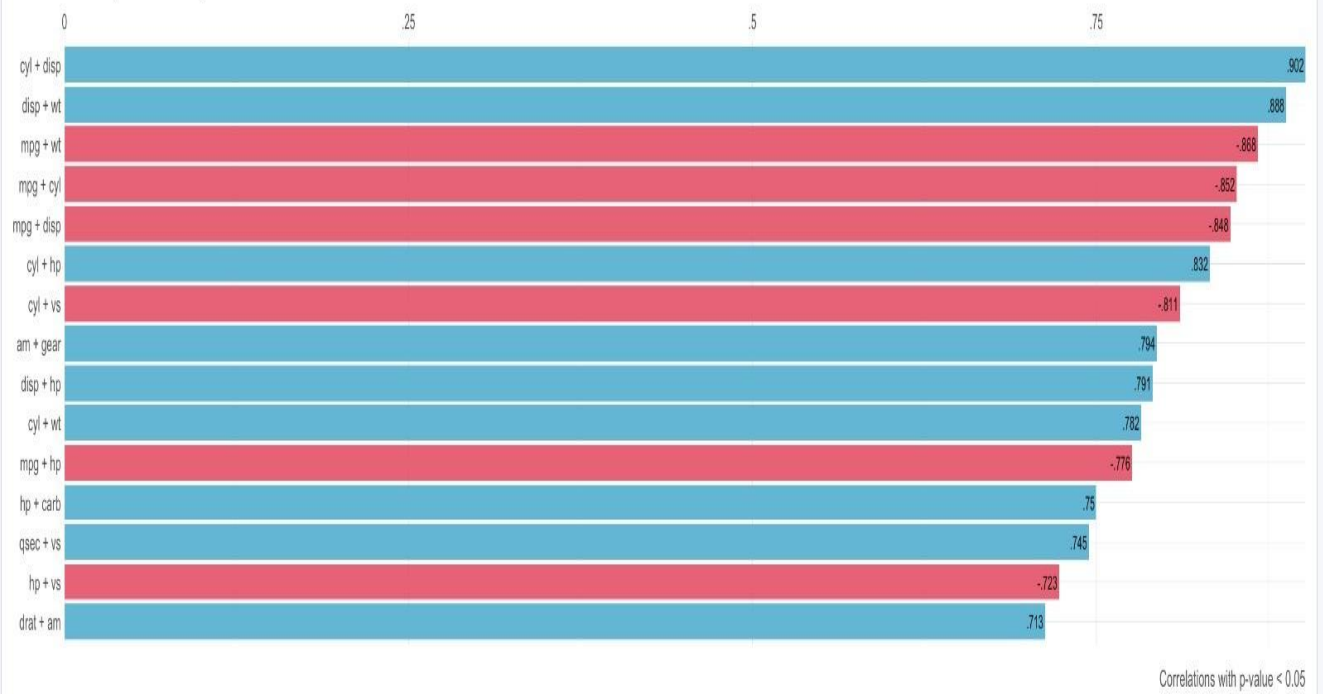
> corr_cross(mtcars, rm.na = T, max_pvalue = 0.05, top = 15, grid = T)
Returning only the top 15. You may override with the 'top' argument
There were 50 or more warnings (use warnings() to see the first 50)
>

```



Ranked Cross-Correlations

15 most relevant (NAs removed)



Date -

Experiment - 10

Aim - Write a program to draw a scatter diagram, residual plots, outliers leverage and influential data points in R.

Code

```
# Load Required Libraries
library(ggplot2)

# Generate Sample Data
set.seed(123)
x <- rnorm(100)
y <- 2 * x + rnorm(100)
data <- data.frame(x, y)

# Scatter Plot
ggplot(data, aes(x = x, y = y)) +
  geom_point(color = "blue") +
  labs(title = "Scatter Diagram", x = "X", y = "Y") +
  theme_minimal()

# Fit Linear Model
model <- lm(y ~ x, data = data)

# Residual Plot (from base R)
plot(model, which = 1, main = "Residuals vs Fitted")

# Identify Outliers (based on residuals)
residuals_sd <- sd(model$residuals)
outliers <- which(abs(model$residuals) > 2 * residuals_sd)
cat("Outlier indices based on residuals:\n")
print(outliers)

# Identify Leverage Points
leverage <- hatvalues(model)
leverage_threshold <- 2 * (ncol(data)) / nrow(data)
leverage_points <- which(leverage > leverage_threshold)
cat("Leverage points (threshold =", round(leverage_threshold, 3), "):\n")
print(leverage_points)

# Identify Influential Points (Cook's Distance)
cooks_d <- cooks.distance(model)
```

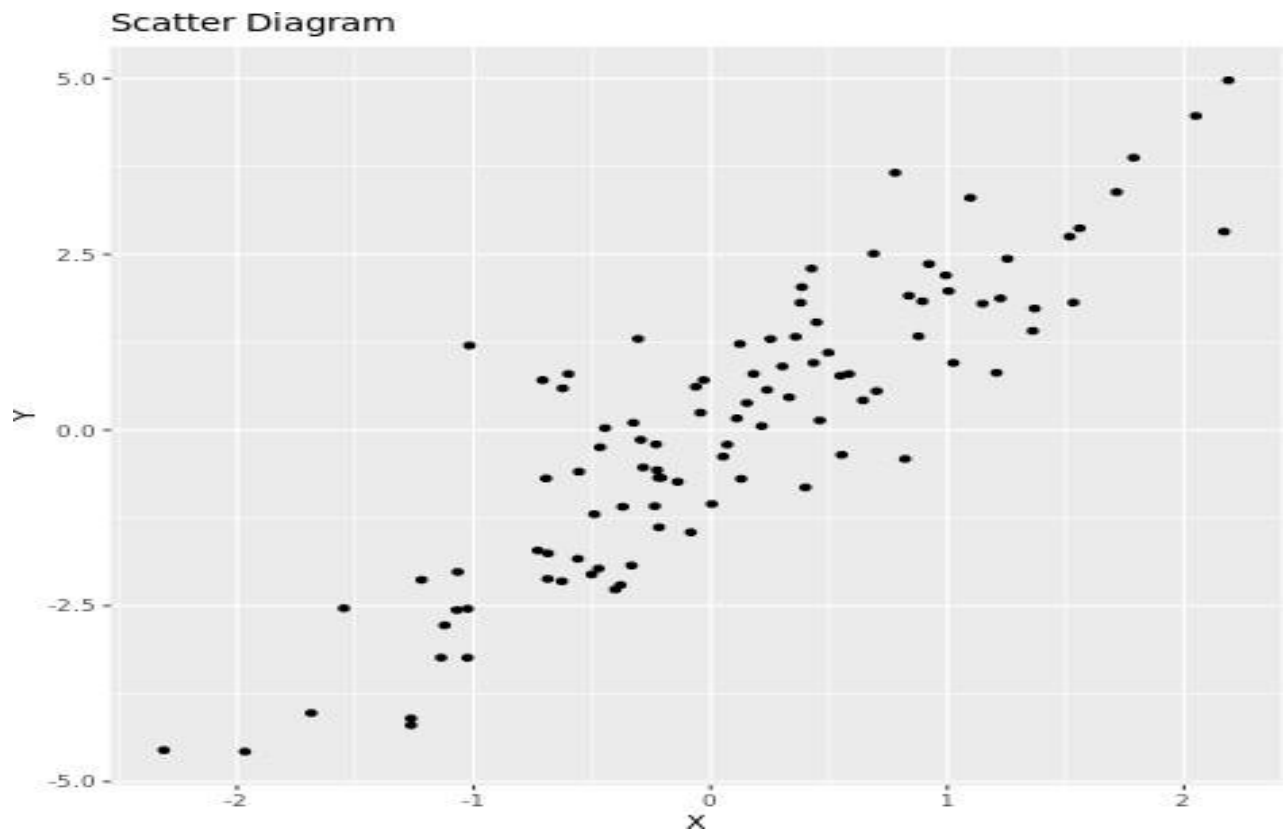
```

cooks_threshold <- 4 / nrow(data)
influential_points <- which(cooksd > cooks_threshold)
cat("Influential points (Cook's distance >",
    round(cooks_threshold, 3), "):\n")
print(influential_points)

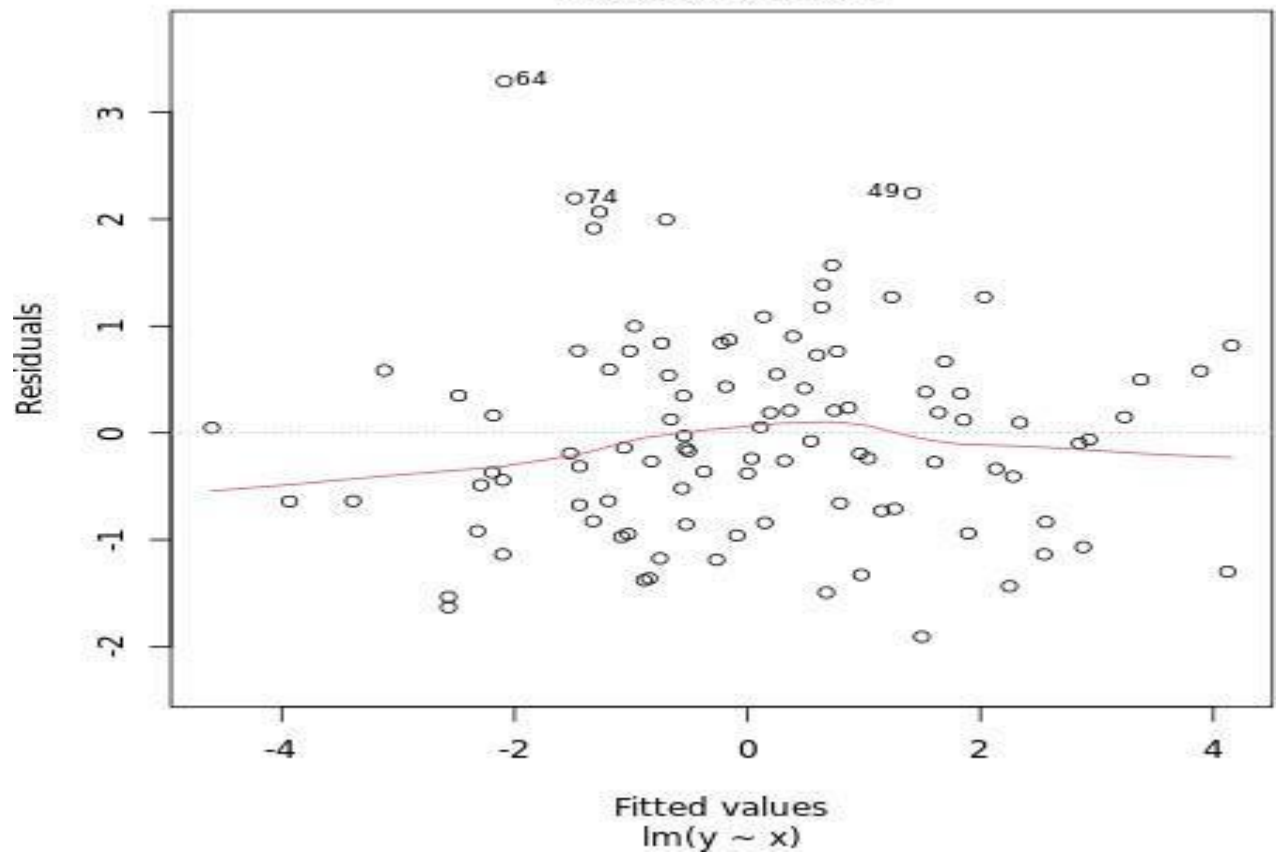
# Optional: Plot Cook's Distance
plot(cooksd, type = "h", main = "Cook's Distance",
     ylab = "Cook's Distance", xlab = "Index")
abline(h = cooks_threshold, col = "red", lty = 2)

```

Output



Residual Plot
Residuals vs Fitted



Date -

Experiment – 09

Aim - Exercises to find the most risky project out of two mutually exclusive projects in SPSS.

Steps -

1. Open the data in SPSS

Launch SPSS and open the data file containing the "Project," "Cost Estimate," and "Time Estimate" variables.

2. Analyse Cost Estimates

- Go to Analyze > Descriptive Statistics > Frequencies.
- Select the "Cost Estimate" variable.
- Click "Statistics" and check "Mean," "Median," and "Standard Deviation."
- Click "OK" to run the analysis.

Examine the output

- Compare the mean, median, and standard deviation of cost estimates for Project A and Project B.
- A higher standard deviation indicates greater cost variability and potential risk.

3. Analyse Time Estimates

- Repeat the same steps as in Step 2, but select the "Time Estimate" variable instead.

Examine the output

- Compare the mean, median, and standard deviation of time estimates for both projects.
- A higher standard deviation suggests greater uncertainty in project completion time, potentially increasing risk.

4. Calculate Cost-Risk

- Create a new variable named "Cost Risk" by multiplying the "Cost Estimate" by a risk factor.
- Choose a risk factor based on your project context (e.g., 1.1 for moderate risk, 1.2 for high risk).
- For example, if using a risk factor of 1.1:
Create a new computed variable named "Cost Risk" with the following formula:
$$\text{Cost Risk} = \text{Cost Estimate} * 1.1$$

5. Calculate Schedule Risk

- Create a new variable named "Schedule Risk" by multiplying the "Time Estimate" by the chosen risk factor.

- For example, using the same risk factor of 1.1:
Create a new computed variable named "Schedule Risk" with the following formula: $\text{Schedule Risk} = \text{Time Estimate} * 1.1$

6. Compare Project Risks

- Calculate the total risk score for each project by summing the "Cost Risk" and "Schedule Risk" values.
- Compare the total risk scores of Project A and Project B.
- The project with the higher total risk score is considered riskier.

Data

Project	Cost_Estimate	Time_Estimate	Cost_Risk	Schedule_Risk	Total_Risk_Score
Project A	100000	12	110000	13.2	110013.2
Project A	120000	14	132000	15.4	132015.4
Project A	110000	13	121000	14.3	121014.3
Project B	80000	10	88000	11	88011
Project B	90000	11	99000	12.1	99012.1
Project B	100000	12	110000	13.2	110013.2

Output

Frequencies

Statistics

VAR00002

N	Valid	6
	Missing	0
Mean		100000.0000
Median		100000.0000
Std. Deviation		14142.13562

VAR00002					
		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	80000.00	1	16.7	16.7	16.7
	90000.00	1	16.7	16.7	33.3
	100000.00	2	33.3	33.3	66.7
	110000.00	1	16.7	16.7	83.3
	120000.00	1	16.7	16.7	100.0
Total		6	100.0	100.0	

➔ Frequencies

Statistics

VAR00003

N	Valid	6
	Missing	0
Mean		12.0000
Median		12.0000
Std. Deviation		1.41421

VAR00003

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	10.00	1	16.7	16.7	16.7
	11.00	1	16.7	16.7	33.3
	12.00	2	33.3	33.3	66.7
	13.00	1	16.7	16.7	83.3
	14.00	1	16.7	16.7	100.0
	Total	6	100.0	100.0	

Date –

Experiment - 08

Aim - Exercises to find the outliers in a dataset in SPSS.

Theory - Identifying outliers in a dataset is crucial for data analysis in SPSS. The algorithm involves the following steps:

Algorithm

Step 1 - Open Dataset containing the variable of interest, such as annual income for individuals.

Step 2 - Access Descriptive Statistics Click on the "Analyze" tab in SPSS.

Step 3 - Navigate to Descriptive Statistics From the Analyze menu, select "Descriptive Statistics."

Step 4 - Select Explore In the submenu, choose "Explore."

Step 5 - Choose Variable Drag the variable (e.g., income) into the box labeled "Dependent List."

Step 6 - Configure Statistics. Click on the "Statistics" button and ensure that the box next to "Percentiles" is checked. Click "Continue."

Step 7 - Generate Box Plot Click "OK" to generate the box plot.

Step 8 - Interpret Box Plot Examine the box plot to identify any circles or asterisks on either end of the box plot. Circles indicate potential outliers, while asterisks indicate extreme outliers.

Step 9 - Calculate Interquartile Range (IQR) Locate the interquartile range (IQR) from the output, typically labeled as "Tukey's Hinges."

Step 10 - Define Outlier Ranges.

Calculate outlier ranges using the formula:

Upper bound: $3\text{rd quartile} + 1.5 * \text{IQR}$

Lower bound: $1\text{st quartile} - 1.5 * \text{IQR}$

Step 11 - Determine Outliers. Identify any values outside the defined outlier ranges. Any data points beyond these ranges are considered outliers.

Output interpretation

1. Box Plot: Examining the box plot visually identifies outliers.
2. Tukey's Hinges: Locate the interquartile range (IQR) from the output.

3. Outlier Ranges: Calculate upper and lower bounds based on the IQR.
4. Identification: Values outside the defined outlier ranges are considered outliers or extreme outliers.

Handling outliers

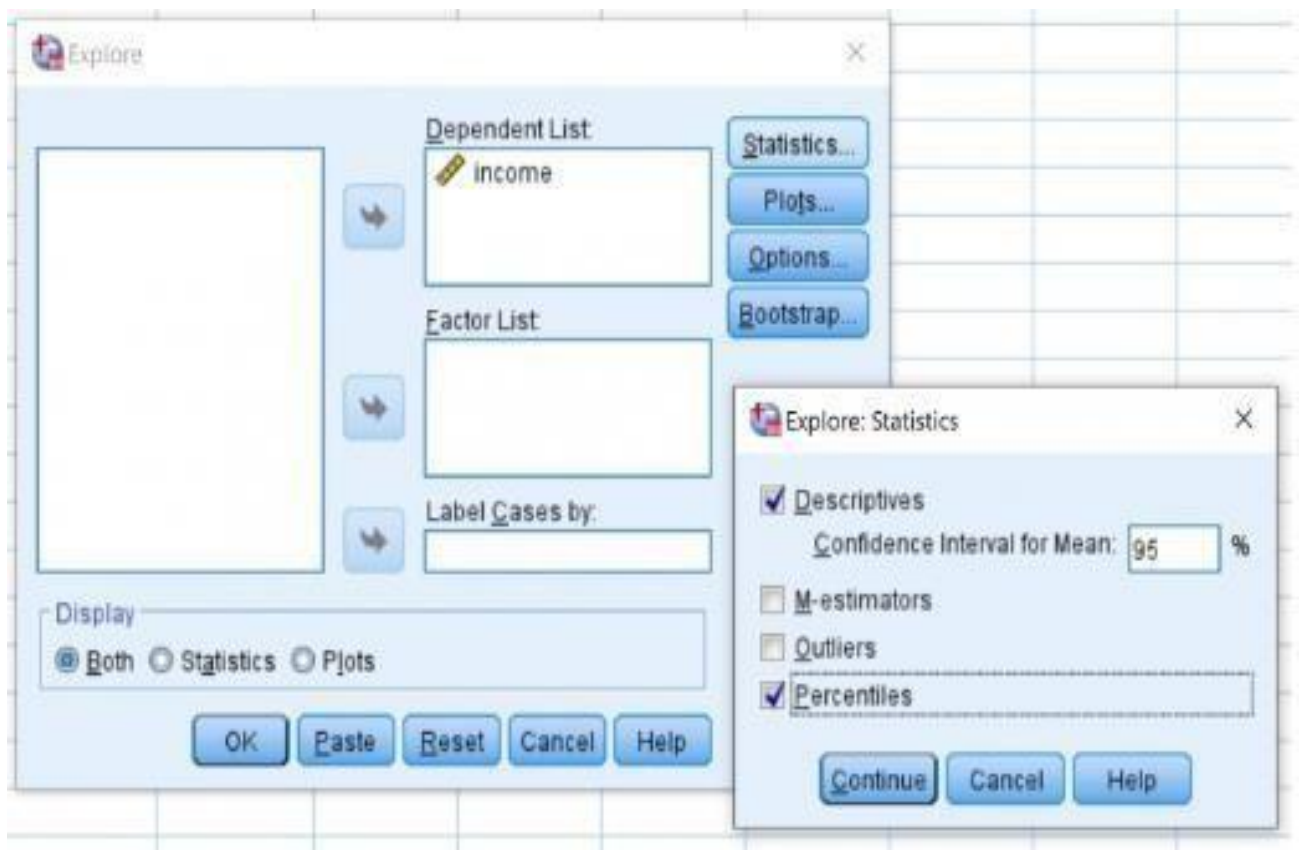
1. Verify Data Entry: Ensure outliers are not the result of data entry errors.
2. Remove Outliers: Consider removing outliers if they significantly impact analysis.
3. Assign New Values: Replace outlier values with appropriate replacements (e.g., mean, median) if they are data entry errors.

Data used

The output includes box plots indicating potential outliers and extreme outliers. Interquartile range and outlier ranges are calculated to identify and handle outliers appropriately.

INCOME	18	24	36	34	38	45	48	54	60	73	79	85	94	98	108
---------------	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

Output

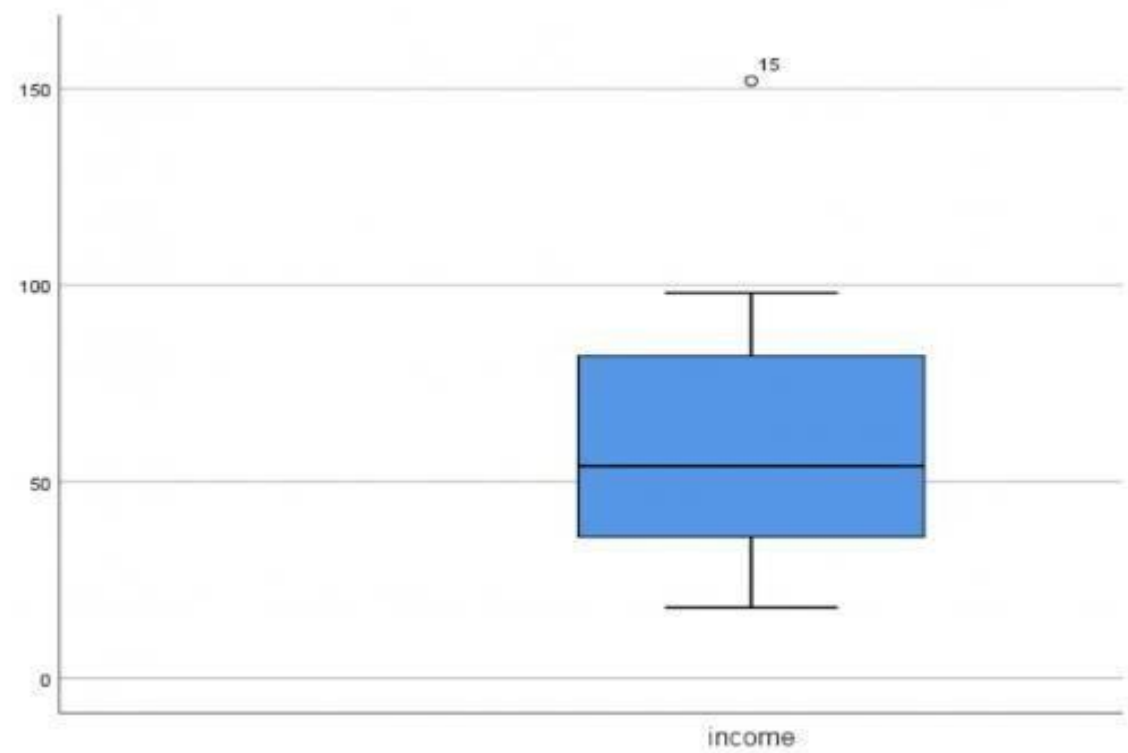


File Edit View Data Transform Analyze Graphs Utilities					
22 :					
	income	var	var	var	
1	18				
2	24				
3	26				
4	34				
5	38				
6	45				
7	48				
8	54				
9	60				
10	73				
11	79				
12	85				
13	94				
14	98				
15	108				
16					
17					
18					

File Edit View Data Transform Analyze Graphs Utilities Extensions Window Help					
22 :					
	income	var			
1	18				
2	24				
3	26				
4	34				
5	38				
6	45				
7	48				
8	54				
9	60				
10	73				
11	79				
12	85				
13	94				
14	98				
15	108				
16					
17					

- Reports
- Descriptive Statistics**
 - Frequencies...
 - Descriptives...
 - Explore...**
 - Crosstabs...
 - Ratio...
 - P-P Plots...
 - Q-Q Plots...
- Bayesian Statistics
- Tables
- Compare Means
- General Linear Model
- Generalized Linear Models
- Mixed Models
- Correlate
- Regression
- Loglinear
- Neural Networks
- Classify
- Dimension Reduction
- Scale
- Nonparametric Tests
- Forecasting
- Survival
- Multiple Response
- Missing Value Analysis...

income



Date –

Experiment – 07

Aim - Exercises to present the data as a frequency table in SPSS.

Theory - Frequency tables in SPSS provide a structured way to summarize categorical data, aiding in data interpretation and analysis. The algorithm to create a frequency table involves several steps within the SPSS software:

AI

Step 1: Open the dataset containing the variable of interest.

Step 2: Access Frequency Analysis. Click on the "Analyze" tab in SPSS.

Step 3: Navigate to Descriptive Statistics From the Analyze menu, select "Descriptive Statistics."

Step 4: Select Frequencies In the submenu, choose "Frequencies."

Step 5: Choose Variable in the Frequencies dialog box, locate the variable for which you want to create a frequency table.

Step 6: Add Variable & Drag the variable from the left panel (Variables) to the right panel (Variable(s)) in the Frequencies dialog box.

Step 7: Optional: Additional Statistics. If desired, click on the "Statistics" button to include additional descriptive statistics such as mean, median, etc.

Step 8: Generate Frequency Table. Click on the "OK" button to generate the frequency table.

Output interpretation

- **Validity Information:** The generated output provides information about the number of valid and missing values for the selected variable.
- **Frequency Table:** The frequency table displays each unique value of the variable, along with its frequency count and percentage.
- ✓ The "Frequency" column indicates how many times each unique value occurs in the dataset.
- ✓ The "Percent" column shows the percentage of each value relative to the total number of valid responses.
- ✓ The sum of the percentages equals 100%.

Data used

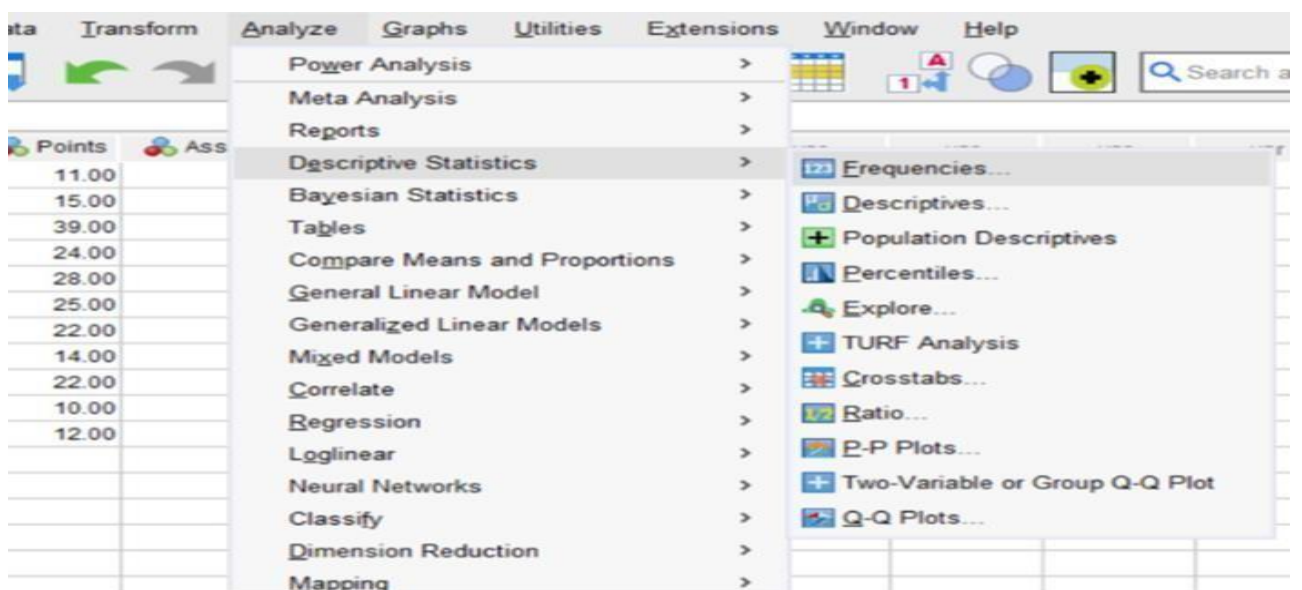
The frequency table output provides a clear summary of the distribution of values within the variable, facilitating easy interpretation and analysis.

- The team name Mavs occurs 4 times, which represents $4/11 = 36.4\%$ of all values in the Team column.
- The team name Rockets occurs 3 times, which represents $3/11 = 27.3\%$ of all values in the Team column.
- The team name Spurs occurs 2 times, which represents $2/11 = 18.2\%$ of all values in the Team column.
- The team name Warriors occurs 2 times, which represents $2/11 = 18.2\%$ of all values in the Team column.

Note that the values in the Percent column add up to 100%.

Output

	Team	Points	Assists	var	
1	Mavs	11.00	11.00		
2	Mavs	15.00	5.00		
3	Rockets	39.00	4.00		
4	Mavs	24.00	3.00		
5	Spurs	28.00	3.00		
6	Rockets	25.00	8.00		
7	Warriors	22.00	4.00		
8	Spurs	14.00	1.00		
9	Rockets	22.00	9.00		
10	Mavs	10.00	2.00		
11	Warriors	12.00	12.00		
12					
13					
14					
15					



 Frequencies X

Points

Assists

Variable(s):

Team

[Statistics...](#)

[Charts...](#)

[Format...](#)

[Style...](#)

[Bootstrap...](#)



☒ Display frequency tables

☐ Create APA style tables

OK
Paste
Reset
Cancel
Help

➔ **Frequencies**

Statistics

Team		
N	Valid	11
	Missing	0

Team

		Frequency	Percent	Valid Percent	Cumulative Percent
Valid	Mavs	4	36.4	36.4	36.4
	Rockets	3	27.3	27.3	63.6
	Spurs	2	18.2	18.2	81.8
	Warriors	2	18.2	18.2	100.0
	Total	11	100.0	100.0	